

SmartRide NEMT — Supervisor Database

Q&A;

FYP Viva Preparation | Database Design & Implementation | 120 Questions with Answers

#	Section	Questions
1	Schema Design & Normalization	15
2	Relationships & Foreign Keys	10
3	Indexing & Query Performance	10
4	PostgreSQL & PostGIS Specific	10
5	Alembic Migrations	10
6	ENUM Types	8
7	Security & Data Integrity	10
8	Transactions & Concurrency	8
9	Analytics & Reporting	8
10	Scalability & Future Design	10
11	ML / Triage Integration	7
12	Real-Time & WebSocket	5
	TOTAL	111

1. Schema Design & Normalization

Q1.

Why did you choose UUID as primary key instead of auto-increment integer? What are the trade-offs?

Answer:

UUIDs prevent enumeration attacks (attacker can't guess IDs), work across distributed systems without collision, and allow client-side ID generation before DB insert. Trade-off: 16 bytes vs 4 bytes, slightly slower B-tree indexing due to random insertion. For an NEMT app with multiple services, UUID is the right choice.

Q2.

Is your database schema in 3NF (Third Normal Form)? Identify any violations if present.

Answer:

Mostly yes. Each table has a single primary key and non-key columns depend only on the PK. Potential concern: `users.role` (active role) could be derived from user_roles` — this is a controlled denormalization for read performance, not a true 3NF violation since it stores 'active' vs 'held' semantics.`

Q3.

Why is `users.role` kept alongside the user_roles` junction table? Isn't that redundant?`

Answer:

Not redundant — they serve different purposes. `user_roles` stores ALL roles a user holds. users.role` stores the ACTIVE portal the user is currently using. A user can hold both patient and driver roles but only be active in one at a time. This distinction drives UI rendering and JWT payload.`

Q4.

Why does `patients` have a separate table instead of storing patient fields directly in users`?`

Answer:

Separation of concerns. Not all users are patients. A driver-only account has no need for `mobility_needs` or emergency_contact`. This avoids NULL columns on non-patient users and follows the Single Responsibility Principle. It also allows the patient profile to evolve independently of auth data.`

Q5.

The `drivers` table has both current_lat/current_lng` and a location` GEOGRAPHY column. Why store it twice?`

Answer:

`current_lat/current_lng` are simple floats exposed to the Flutter app via JSON API — easy to serialize. location` is a PostGIS GEOGRAPHY type used exclusively for spatial SQL queries (ST_DWithin, ST_Distance). A trigger keeps them in sync automatically. Dual storage avoids spatial type conversion overhead in the API layer.`

Q6.

Why are `pickup_lat` and pickup_lng` stored as separate DOUBLE PRECISION columns instead of a GEOGRAPHY type?`

Answer:

Pickup location is a one-time snapshot captured at ride request — it is never spatially queried after creation. Converting it to GEOGRAPHY would add overhead with no benefit. Driver and hospital locations are queried spatially repeatedly, justifying the GEOGRAPHY type there.

Q7.

Justify storing ``estimated_fare_pkr`` and ``final_fare_pkr`` as separate columns.

Answer:

``estimated_fare_pkr`` is computed when the ride is requested based on estimated distance. ``final_fare_pkr`` is set when the ride completes based on actual distance and time. They differ due to traffic, route deviation, or wait time. Keeping both allows fare reconciliation, dispute resolution, and analytics on estimation accuracy.

Q8.

Why does ``triage_events`` have ``ON DELETE CASCADE`` from rides?

Answer:

Triage events have no meaning without their parent ride — they are purely derived/dependent data. If a ride is deleted (e.g., test data cleanup, GDPR erasure), orphaned triage events would waste storage and confuse analytics. CASCADE ensures referential integrity automatically.

Q9.

What is the purpose of ``analytics_hourly``? Why not compute analytics from rides table on the fly?

Answer:

Pre-aggregated analytics avoid expensive GROUP BY + COUNT queries on a high-volume rides table at dashboard load time. As rides grow to millions, real-time aggregation becomes slow. ``analytics_hourly`` is populated by a background job (cron), making dashboards O(1) reads instead of O(n) scans.

Q10.

Why does ``hospitals`` store ``ed_capacity`` and ``ed_current_load`` as static integers instead of real-time data?

Answer:

True real-time ED occupancy requires FHIR integration with hospital systems, which may be unavailable or unreliable. Static integers serve as a best-effort approximation updated periodically. The ``fhir_endpoint`` column exists to support future real-time integration when available.

Q11.

Explain the ``specialties`` column in hospitals stored as JSON instead of a separate junction table.

Answer:

Hospital specialties are read-only reference data that rarely change and are always queried as a complete list (never filtered by individual specialty on the hospital table). JSON avoids a JOIN and simplifies the API response. If specialty-based hospital search were needed, a junction table would be the correct choice.

Q12.

Why does ``family_notifications`` store ``recipient_phone`` directly instead of linking to ``patients.emergency_contact_phone``?

Answer:

At send time, the contact phone is captured and frozen. If a patient later changes their emergency contact, historical notification records must still show who was notified. Linking to the live patient column would lose this audit trail. The denormalization is intentional for record integrity.

Q13.

What happens if a driver is deleted — which rides and records are affected?

Answer:

``rides.driver_id`` is a nullable FK without CASCADE. If a driver is deleted, their rides retain the `driver_id` value but the FK becomes a dangling reference — unless a SET NULL or RESTRICT constraint is applied. Best practice: soft-delete drivers (set `is_active=false`) rather than hard-delete to preserve ride history integrity.

Q14.

Why is ``cancel_reason`` TEXT instead of an ENUM?

Answer:

Cancellation reasons are open-ended and vary by context (patient no-show, driver emergency, system error, etc.). An ENUM would require a migration every time a new reason is needed. TEXT allows flexibility. If analytics on cancel reasons are needed, a separate ENUM column or tag system can be added later without losing free-text detail.

Q15.

What is the UNIQUE constraint on ``analytics_hourly(snapshot_hour, city, zone_id)``?

Answer:

Prevents duplicate hourly snapshots for the same city+zone combination. Without it, a cron job that runs twice in the same hour would insert duplicate rows, corrupting aggregate counts. The constraint acts as an upsert guard — the job can use `INSERT ... ON CONFLICT DO UPDATE` safely.

2. Relationships & Foreign Keys

Q16.

Draw and explain the ER diagram for rides, patients, drivers, and hospitals.

Answer:

Patient (1)  < Ride >  (0..1) Driver. Ride >  (0..1) Hospital. One patient can have many rides. A ride may have zero or one driver (none until accepted). A ride may target zero or one hospital. Driver and Hospital each have one-to-many rides. Patient links to User (1:1), Driver links to User (1:1).

Q17.

Why is `driver\_id` nullable in the rides table?

Answer:

When a ride is first created (status=pending), no driver has accepted it yet. Making driver_id NOT NULL would require a driver to be assigned atomically at creation — impossible for a dispatch model where drivers browse and accept available rides. Nullable reflects the real-world state where a ride exists before assignment.

Q18.

Why is `hospital\_id` nullable in the rides table?

Answer:

For scheduled rides, the patient may not yet know the destination hospital. For emergency rides, hospital selection happens after triage assessment. The ride record is created first, hospital assigned after triage. Nullable supports this two-phase creation pattern.

Q19.

Can one user have multiple patient profiles?

Answer:

No. `patients.user\_id` has a UNIQUE constraint, enforcing a strict 1:1 relationship. One user account = one patient profile. The multi-role system allows a user to be both a patient AND a driver, but each role has only one profile per user.

Q20.

What is the purpose of the `uq\_user\_roles\_user\_role` unique constraint?

Answer:

Prevents duplicate role assignments. Without it, a bug or concurrent request could insert the same role twice for a user (e.g., two 'patient' rows for user X). The constraint ensures idempotent role assignment — inserting an existing role fails gracefully rather than creating duplicates.

Q21.

If a patient is deleted, what happens to their rides?

Answer:

Since `rides.patient\_id` is a NOT NULL FK, deleting a patient without CASCADE would fail with a foreign key violation (RESTRICT behavior by default). The correct approach is either: 1) soft-delete patients (is_active=false), or 2) define ON DELETE CASCADE if ride history is acceptable to lose, or 3) ON DELETE SET NULL if rides should be retained anonymously.

Q22.

Why does `user\_roles` have its own UUID PK instead of composite PK?

Answer:

A composite PK (user_id, role) would work functionally, but a surrogate UUID PK is simpler for ORM frameworks, API responses, and future foreign key references. It also makes the unique constraint explicit and separate from the PK, allowing easier debugging and migration.

Q23.

How does the multi-role system work at DB level?

Answer:

1) User registers with initial role(s) → rows inserted into `user\_roles`. 2) `users.role` set to active role. 3) JWT issued with `roles[]` (all) + `active\_role`. 4) User calls `/auth/switch-role` → `users.role` updated to new active role. 5) New JWT issued. RBAC checks query `user\_roles` for held roles, not `users.role`.

Q24.

Can a driver also be a patient? How is this supported?

Answer:

Yes. The `user\_roles` table allows one user_id to have both 'patient' and 'driver' role rows. Both a `patients` record and a `drivers` record can exist for the same user_id. The user switches between portals via role switching. The Flutter app shows either patient or driver UI based on `active\_role` in the JWT.

Q25.

What DB constraint prevents a ride being assigned to an unverified driver?

Answer:

No DB-level constraint exists — this is enforced at the application layer. The `/rides/{id}/accept` endpoint checks `driver.is\_verified == True` before allowing assignment, returning HTTP 403 otherwise. A DB check constraint `CHECK (is\_verified = TRUE)` could be added to `rides` via a trigger for defense-in-depth.

3. Indexing & Query Performance

Q26.

Why is there a GIST index on `drivers.location` and `hospitals.location`?

Answer:

GIST (Generalized Search Tree) indexes support spatial operators like ST_DWithin, ST_Distance, ST_Intersects. A B-tree index cannot index GEOGRAPHY types. GIST enables efficient nearest-neighbor and radius queries — finding the 3 closest available drivers to a pickup point goes from $O(n)$ full scan to $O(\log n)$ with GIST.

Q27.

Explain why `ix\_rides\_status` exists. Write a sample query that benefits from it.

Answer:

Drivers frequently query pending rides: `SELECT * FROM rides WHERE status = 'pending' ORDER BY requested_at DESC`. Without the index, PostgreSQL scans all ride rows. With `ix\_rides\_status`, it jumps directly to 'pending' rows. Critical since most rides are completed — the index isolates the small active subset.

Q28.

Why is `ix\_rides\_requested\_at` created as DESC?

Answer:

Most queries fetch recent rides first (driver dashboard, admin panel, patient history). Descending order matches the most common sort direction, so PostgreSQL can scan the index forward without reversing. An ASC index used for DESC queries forces a backward scan, which is less cache-friendly.

Q29.

Without PostGIS, how would you find the 3 nearest available drivers?

Answer:

Use Haversine formula: $\text{compute distance} = 2R * \arcsin(\sqrt{\sin^2(\Delta\text{lat}/2) + \cos(\text{lat1})\cos(\text{lat2})\sin^2(\Delta\text{lng}/2)})$ for every available driver. This is $O(n)$ — scans all available drivers. For 10,000 drivers, that's 10,000 distance calculations per ride request. PostGIS GIST index reduces this to $O(\log n)$.

Q30.

Write SQL to find the nearest available hospital using PostGIS.

Answer:

```
SELECT id, name, ST_Distance(location, ST_MakePoint(73.0479, 33.6844)::geography) AS dist_m
FROM hospitals WHERE is_active = TRUE ORDER BY location <-> ST_MakePoint(73.0479, 33.6844)::geography LIMIT 1;
```

— The <-> operator uses the GIST index for KNN (k-nearest-neighbor) search.

Q31.

Which table will have the most rows over time? How would you scale it?

Answer:

`rides` — every trip creates a row plus multiple child rows (triage_events, family_notifications, hospital_alerts). Scale strategies: 1) Range partition by requested_at month. 2) Archive completed rides older than 1 year to cold storage. 3) Read replicas for reporting queries. 4) Index-only scans for

dashboard aggregations.

Q32.

Would you add an index on `rides.patient\_id`?

Answer:

Yes — already exists as `ix\_rides\_patient\_id`. Patients frequently view their own ride history: `SELECT * FROM rides WHERE patient_id = $1 ORDER BY requested_at DESC`. Without the index, this scans the entire rides table. The index makes it $O(\log n)$ to locate that patient's rides.

Q33.

What is a covering index? Would any query benefit from one?

Answer:

A covering index includes all columns needed by a query, so PostgreSQL never touches the main table (index-only scan). Example: `CREATE INDEX ON rides(status, requested_at DESC) INCLUDE (id, patient_id, driver_id)`; — The driver dashboard query `SELECT id, patient_id, driver_id FROM rides WHERE status='pending' ORDER BY requested_at DESC` would be satisfied entirely from the index.

Q34.

How does PostgreSQL use `ix\_drivers\_status` when querying available drivers?

Answer:

PostgreSQL's query planner evaluates selectivity. If 'available' drivers are ~10% of total, the planner uses the B-tree index to fetch matching rows directly (bitmap index scan or index scan), then fetches full rows from the heap. If 'available' were 80% of rows, the planner might choose a seq scan instead as it would be faster.

Q35.

Difference between B-tree and GIST indexes.

Answer:

B-tree: default index, works on ordered scalar types (int, text, timestamp). Supports =, <, >, BETWEEN, IN. Cannot index geometric/geographic types. GIST: generalized tree supporting arbitrary types — PostGIS uses it for GEOGRAPHY/GEOMETRY. Supports spatial operators (&&, <->, ST_DWithin). Use B-tree for normal columns, GIST for spatial data or full-text search.

4. PostgreSQL & PostGIS Specific

Q36.

Why PostgreSQL over MySQL or SQLite?

Answer:

PostgreSQL supports: 1) PostGIS extension for spatial queries — critical for finding nearest drivers/hospitals. 2) Native UUID type and `uuid_generate_v4()`. 3) JSONB for semi-structured data. 4) Advanced ENUMs as first-class types. 5) Async-compatible drivers (asyncpg). SQLite lacks PostGIS and is not production-grade for concurrent writes. MySQL has weaker PostGIS support.

Q37.

What is PostGIS and why use it over Haversine?

Answer:

PostGIS is a PostgreSQL extension adding spatial objects, functions, and indexes. It implements OGC standards. Benefits over manual Haversine: 1) GIST spatial indexes — $O(\log n)$ vs $O(n)$. 2) Built-in functions (`ST_Distance`, `ST_DWithin`, `ST_MakePoint`). 3) Handles edge cases (poles, antimeridian). 4) Supports SRID projections. 5) Used by production mapping systems worldwide.

Q38.

Explain GEOGRAPHY(POINT, 4326). What does 4326 mean?

Answer:

GEOGRAPHY is a PostGIS type that stores coordinates on the Earth's curved surface (as opposed to flat GEOMETRY). POINT specifies a single coordinate pair. 4326 is the SRID (Spatial Reference ID) for WGS84 — the GPS coordinate system used by all modern devices and Google Maps. Distance calculations in GEOGRAPHY are in meters on the spheroid, not flat 2D space.

Q39.

What is the `uuid-oss` extension?

Answer:

A PostgreSQL extension providing UUID generation functions. `uuid_generate_v4()` generates cryptographically random UUIDs (version 4). It is used as the DEFAULT for all primary key columns: `id UUID DEFAULT uuid_generate_v4()`. Without this extension, UUIDs must be generated in application code before insertion.

Q40.

What does trigger `trg\_driver\_location` do? Why not compute in app code?

Answer:

The trigger fires on INSERT or UPDATE of `current_lat`/`current_lng`` on the drivers table, automatically setting `location = ST_MakePoint(current_lng, current_lat)::geography``. This ensures the GEOGRAPHY column is always consistent with the scalar columns without requiring every update path in the application to remember to compute it. A single source of truth enforced at DB level.

Q41.

Difference between GEOGRAPHY and GEOMETRY in PostGIS.

Answer:

GEOMETRY assumes a flat 2D plane — distances are in the coordinate unit (degrees for WGS84), causing significant errors at large distances. GEOGRAPHY assumes Earth's curved surface

(spheroid) — distances are in meters and accurate globally. Use GEOGRAPHY when accuracy matters and data covers large areas. GEOMETRY is faster and better for small, locally-projected areas.

Q42.

Write SQL to find hospitals within 10km of (33.6844, 73.0479).

Answer:

```
SELECT id, name, ST_Distance(location, ST_MakePoint(73.0479, 33.6844)::geography) AS dist_m
FROM hospitals WHERE ST_DWithin(location, ST_MakePoint(73.0479, 33.6844)::geography,
10000) AND is_active = TRUE ORDER BY dist_m; — Note: ST_MakePoint takes (longitude, latitude)
order. 10000 = 10km in meters.
```

Q43.

What is JSONB vs JSON in PostgreSQL? Why use JSONB for `hospital\_alerts.payload`?

Answer:

JSON stores the raw text string as-is. JSONB stores a decomposed binary representation. JSONB advantages: 1) Faster reads (no re-parsing). 2) Supports GIN indexes for key/value queries. 3) Deduplicates keys. 4) Allows -> and ->> operators. For `payload` which may be queried for debugging/retries, JSONB is preferred. The write overhead is negligible.

Q44.

What is TIMESTAMPTZ vs TIMESTAMP? Why use TIMESTAMPTZ?

Answer:

TIMESTAMP stores date/time with no timezone info — ambiguous when servers move regions or DST changes. TIMESTAMPTZ stores UTC internally and converts to session timezone on display. For an app used in Pakistan (PKT, UTC+5), all times stored as UTC ensure correct comparisons, sorting, and analytics regardless of server timezone. Always use TIMESTAMPTZ in production.

Q45.

What does `alembic\_version` table store?

Answer:

A single-row table storing the current migration revision string (e.g., '0004'). Alembic reads this to determine which migrations have been applied and which are pending. It is managed entirely by Alembic — you should never edit it manually. It does not appear in SQLAlchemy models because it is a migration-tooling artifact, not application data.

5. Alembic Migrations

Q46.

What is Alembic and why use migrations?

Answer:

Alembic is a database migration tool for SQLAlchemy. It tracks schema changes as versioned Python scripts with `upgrade()` and `downgrade()` functions. Benefits: 1) Schema changes are code-reviewed and version-controlled. 2) Reproducible across dev/staging/prod. 3) Rollback support. 4) Team members can apply changes with ``alembic upgrade head`` without knowing the raw SQL.

Q47.

Walk through all 4 migrations.

Answer:

0001: Initial schema — 8 tables (users, patients, drivers, hospitals, rides, triage_events, hospital_alerts, family_notifications, analytics_hourly), 6 ENUMs, PostGIS setup, GIST indexes, triggers, seeds 5 Islamabad hospitals. | 0002: Adds user_roles junction table for multi-role support; backfills existing users' roles from users.role. | 0003: Adds wallet_balance_pkr DOUBLE PRECISION DEFAULT 0 to drivers. | 0004: Adds patient_rating NUMERIC(2,1) and rating_comment TEXT to rides.

Q48.

Why was `wallet\_balance\_pkr` added in 0003 instead of the initial schema?

Answer:

The wallet feature was designed iteratively — the initial MVP focused on ride dispatch and triage. Wallet/payment tracking was a Phase 2 feature added after core functionality was validated. This is normal iterative development: schema evolves as product requirements are confirmed, avoiding over-engineering the initial schema.

Q49.

How does Alembic track applied migrations?

Answer:

Via the ``alembic_version`` table in the database. Each migration has a unique revision ID. After ``alembic upgrade head`` runs migration 0004, the table stores '0004'. Running ``alembic upgrade head`` again is a no-op. Running ``alembic downgrade -1`` removes 0004's changes and sets the version back to '0003'.

Q50.

What would happen if you ran `alembic downgrade -1` on production?

Answer:

It would execute the `downgrade()` function of the latest migration. For 0004, this drops ``patient_rating`` and ``rating_comment`` columns — ALL existing rating data is permanently lost. For 0003, `wallet_balance_pkr` is dropped — all wallet data lost. This is irreversible without a backup. Never run downgrade on production without a verified DB backup.

Q51.

Risk of migration 0002 backfilling existing users.

Answer:

The backfill inserts rows into `user_roles` for all existing users based on their current `users.role` value. Risks: 1) Long-running migration on large user tables can lock the table. 2) If a user has a NULL role, the backfill may fail. 3) If run during peak traffic, the INSERT statements compete with live writes. Mitigation: run during maintenance window, use batched inserts.

Q52.

How to add ``trip_distance_km`` to `rides` using Alembic.

Answer:

1) Run: `alembic revision --autogenerate -m 'add_trip_distance_to_rides'`. 2) Review generated file — verify `upgrade()` has: `op.add_column('rides', sa.Column('trip_distance_km', sa.Numeric(8,3), nullable=True))`. 3) Verify `downgrade()` has: `op.drop_column('rides', 'trip_distance_km')`. 4) Run: `alembic upgrade head`.

Q53.

What is the ``alembic_version`` table?

Answer:

Single-row PostgreSQL table managed by Alembic. Stores the current head revision ID string. Alembic reads it before every upgrade/downgrade to know the current state. It is created automatically by ``alembic upgrade head`` if it doesn't exist. It is not part of the SQLAlchemy models — it's tooling infrastructure.

Q54.

Can two developers create conflicting migrations?

Answer:

Yes — if Dev A creates `0005_feature_x` and Dev B creates `0005_feature_y` independently, both with the same revision number, Alembic will reject the conflict. Resolution: use ``alembic merge`` to create a merge migration that has both as parents, or renumber one migration. Modern teams use branch-based migration naming and run CI checks to catch conflicts.

Q55.

Why never edit a migration that ran on production?

Answer:

Once a migration runs on production, `alembic_version` records its revision ID as applied. If you edit the file, the production DB still thinks that revision is complete — but the actual schema may differ from what the edited file would produce. This causes drift between migration history and actual schema, breaking future upgrades and making rollbacks unreliable.

6. ENUM Types

Q56.

Why use PostgreSQL native ENUMs instead of VARCHAR with CHECK constraint?

Answer:

PostgreSQL ENUMs: 1) Stored as integers internally — 4 bytes vs VARCHAR overhead. 2) Enforced at DB level — no invalid values possible even from direct SQL access. 3) Self-documenting schema — ENUM values visible in `pg_type`. 4) Type-safe in SQLAlchemy. CHECK constraints on VARCHAR work but allow any string that passes the check, not true type safety. Trade-off: adding ENUM values requires a migration.

Q57.

List all ride_status values and valid transitions.

Answer:

pending → driver_assigned → driver_en_route → patient_picked_up → arrived_at_hospital → completed. From any status → cancelled (except completed). pending: no driver yet. driver_assigned: driver accepted. driver_en_route: driver heading to patient. patient_picked_up: patient in vehicle. arrived_at_hospital: at destination. completed: ride finished, fare settled.

Q58.

Can a ride go from `completed` back to `driver\_en\_route`?

Answer:

No DB-level ENUM constraint prevents this — ENUMs only validate that the value is valid, not that the transition is valid. State transition rules are enforced at the application layer (FastAPI service). A CHECK constraint or PostgreSQL trigger could enforce valid transitions at DB level for defense-in-depth, but this is not currently implemented.

Q59.

What happens if you insert an invalid ride_type like 'urgent'?

Answer:

PostgreSQL raises: ERROR: invalid input value for enum ride_type: 'urgent'. The INSERT fails and the transaction is rolled back. This is the advantage of native ENUMs — invalid values are rejected at the DB level regardless of how the data was sent (API, direct SQL, migration script).

Q60.

How to add a new value to a PostgreSQL ENUM without downtime?

Answer:

ALTER TYPE ride_type ADD VALUE 'priority'; — In PostgreSQL 9.1+, adding a value to an ENUM does not require a full table rewrite and can be done online with minimal locking. However, the new value is only visible after the current transaction commits. Note: you cannot remove or rename ENUM values without dropping and recreating the type (requires a migration and downtime).

Q61.

Why is `severity\_level` ENUM '1'-5' instead of INTEGER?

Answer:

ENUM '1'-5' provides: 1) DB-level validation — only values 1-5 accepted. 2) Semantic clarity in schema. INTEGER would accept 0, 99, -1. A CHECK constraint (INTEGER CHECK (severity_level

BETWEEN 1 AND 5)) achieves similar validation but ENUM is cleaner for a fixed small set. Trade-off: ENUM '1'-5' looks odd; INTEGER with CHECK is arguably more Pythonic.

Q62.

Why does `driver\_status` have 3 values instead of a boolean?

Answer:

A boolean `is\_online` cannot distinguish between 'available for new rides' and 'currently on a ride'. The 3 states are semantically distinct: offline=not working, available=waiting for rides, busy=currently on active ride. This distinction is critical for dispatch — only 'available' drivers are shown pending rides, not 'busy' ones.

Q63.

Risk of hardcoding medical specialties in an ENUM.

Answer:

Medical taxonomy evolves — new specialties emerge, names change. Adding 'interventional_radiology' requires an Alembic migration and deployment. ENUMs cannot be altered without downtime on older PostgreSQL. For a production healthcare app, a `specialties` reference table (with id, name, is_active) would be more flexible, allowing additions via INSERT without schema changes.

7. Security & Data Integrity

Q64.

Where are passwords stored? What hashing algorithm should be used?

Answer:

Stored in `users.password\_hash` as a hashed string. Should use bcrypt or Argon2id. Never MD5 or SHA-1 — they are fast hash functions, allowing brute-force attacks at billions of hashes/second. bcrypt is slow by design (cost factor adjustable), making brute-force impractical. Argon2id (winner of Password Hashing Competition) is the modern standard, resistant to GPU and side-channel attacks.

Q65.

The `fcm\_token` in users is stored in plaintext. What security risk does this pose?

Answer:

FCM tokens allow sending push notifications to a user's device. If the DB is breached, attackers could send malicious notifications to all users. Mitigation: 1) FCM tokens are not secrets per se — they are rotated by Firebase. 2) Encrypt at rest if compliance requires it. 3) Ensure DB access is strictly controlled. 4) Tokens expire and rotate automatically.

Q66.

How does your schema prevent a ride being assigned to an unverified driver?

Answer:

No DB constraint — enforced at application layer. The ride acceptance endpoint checks `driver.is\_verified == True`. A stronger approach: add a DB-level trigger or CHECK that prevents driver_id assignment when the referenced driver has is_verified=false. Defense-in-depth: validate at API layer + DB layer.

Q67.

What PII does your database store? How is it protected?

Answer:

PII stored: full_name, date_of_birth, emergency_contact_name/phone (patients), phone number (users), license_no (drivers), GPS coordinates (rides). Protection: 1) Password hashed. 2) DB access via connection string in environment variables (not hardcoded). 3) JWT for API auth. 4) For GDPR/local compliance: consider encrypting DOB and contact fields at rest.

Q68.

Is there any audit trail for sensitive actions?

Answer:

Partial — `cancelled\_at`, `completed\_at`, `driver\_assigned\_at` timestamps provide a timeline. However, there is no formal audit log table tracking WHO changed WHAT and WHEN beyond timestamps. Best practice: add an `audit\_log` table (user_id, action, table_name, row_id, old_value, new_value, created_at) and populate via triggers for sensitive operations.

Q69.

What is SQL injection? Show vulnerable vs safe query.

Answer:

SQL injection: attacker injects SQL code via user input. Vulnerable: query = f"SELECT * FROM users WHERE phone = '{user_input}'" — if user_input = "" OR '1'=1", returns all users. Safe

(parameterized): `db.execute(text('SELECT * FROM users WHERE phone = :phone'), {'phone': user_input})` — SQLAlchemy/asyncpg always use parameterized queries, preventing injection.

Q70.

How does CASCADE DELETE affect integrity when a ride is deleted?

Answer:

Deleting a ride cascades to: `triage_events`, `hospital_alerts`, `family_notifications` — all related records deleted. `rides.patient_id` and `rides.driver_id` have no cascade — they are FKs pointing TO patients/drivers, not the other way. So deleting a ride does NOT affect the patient or driver records. The cascade direction is child→parent-initiated-deletion only.

Q71.

Should `patients.date_of_birth` be encrypted at rest?

Answer:

Yes for a production healthcare app. DOB is PII and in combination with name and phone, is sufficient for identity theft. HIPAA (US) and local Pakistani data protection laws would require it. Options: 1) Application-level encryption (AES-256) before storing. 2) PostgreSQL `pgcrypto` extension. 3) Transparent Data Encryption at storage level. The current schema stores it in plaintext — a compliance gap for a real deployment.

Q72.

How to implement row-level security ensuring patients see only their own rides?

Answer:

PostgreSQL RLS: `ALTER TABLE rides ENABLE ROW LEVEL SECURITY; CREATE POLICY patient_own_rides ON rides USING (patient_id = current_setting('app.current_patient_id')::uuid);` — The application sets `app.current_patient_id` at session start. All queries on rides automatically filter to matching rows. Drivers/admins get separate policies. RLS is enforced at DB level, independent of application logic.

Q73.

What injection risk could `hospital_alerts.payload` JSONB introduce?

Answer:

JSONB stores structured data — not executed as code, so no SQL injection risk from JSONB content itself. Risk: if the payload is forwarded to external systems (webhooks, FHIR endpoints) without sanitization, it could contain crafted values causing XSS in web UIs or injection in downstream systems. Validate and sanitize all fields before forwarding to external APIs.

8. Transactions & Concurrency

Q74.

Two drivers accept the same ride simultaneously. How does your schema handle this?

Answer:

Atomic UPDATE pattern: `UPDATE rides SET driver_id=$1, status='driver_assigned', driver_assigned_at=NOW() WHERE id=$2 AND driver_id IS NULL` — Only ONE driver wins: the first to execute gets `driver_id` set, the second finds `driver_id IS NULL` = false and updates 0 rows. The API returns 409 Conflict when `rowcount=0`. PostgreSQL's row-level locking ensures atomicity — no two UPDATES modify the same row simultaneously.

Q75.

Explain the atomic `UPDATE ... WHERE driver_id IS NULL` pattern.

Answer:

This is an optimistic locking pattern using a conditional UPDATE. The condition `driver_id IS NULL` acts as a version check. PostgreSQL acquires a row-level exclusive lock during the UPDATE. If two transactions attempt simultaneously, one proceeds and the other waits, then finds the condition false (`driver_id` is no longer NULL) and updates 0 rows. The application checks affected rowcount to detect the race.

Q76.

What is a database transaction? Give a critical example.

Answer:

A transaction is a set of DB operations that execute atomically — all succeed or all fail. Critical example: when a ride completes: 1) `UPDATE rides SET status='completed', final_fare_pkr=X`. 2) `UPDATE drivers SET wallet_balance_pkr = wallet_balance_pkr + X`. Both must succeed together. If step 2 fails after step 1 commits, the driver loses money. Wrapping in a transaction ensures rollback if either step fails.

Q77.

Optimistic vs pessimistic locking. Which does your system use?

Answer:

Pessimistic: acquire a lock before reading (`SELECT FOR UPDATE`) — blocks other transactions. Optimistic: no lock on read; check a condition on write (`driver_id IS NULL`). Your system uses optimistic locking for ride assignment — no `SELECT FOR UPDATE`, just a conditional UPDATE. This is better for high-concurrency dispatch since it avoids blocking drivers from viewing the rides list.

Q78.

What isolation level does PostgreSQL use by default?

Answer:

READ COMMITTED — a transaction sees only data committed before each statement executes (not before the transaction started). This means two reads in the same transaction can see different data if another transaction commits between them. For ride assignment, this is acceptable since the atomic UPDATE handles the race. SERIALIZABLE isolation would be needed for more complex financial transactions.

Q79.

Server crashes after driver assigned but before patient notified — what DB state?

Answer:

The DB is consistent — the UPDATE committed (driver assigned, status=driver_assigned). The crash only affects the in-flight WebSocket/push notification. On recovery, the app should detect rides in driver_assigned status with no notification sent and retry notification. This is why status timestamps (driver_assigned_at) matter — they allow reconciliation after failure.

Q80.

What is a deadlock? Can it occur in your schema?

Answer:

Deadlock: Transaction A holds lock on row 1, waits for row 2. Transaction B holds lock on row 2, waits for row 1. Neither can proceed. In your schema: possible if two operations update both rides and drivers in opposite orders (e.g., ride completion updates rides then drivers, while another operation updates drivers then rides). PostgreSQL detects deadlocks and aborts one transaction with ERROR 40P01. Prevention: always acquire locks in consistent order.

Q81.

How is `wallet\_balance\_pkr` updated when ride completes? Is this atomic?

Answer:

Should be atomic: within a single transaction — UPDATE rides SET status='completed', final_fare_pkr=X WHERE id=\$1; then UPDATE drivers SET wallet_balance_pkr = wallet_balance_pkr + X WHERE id=\$2. Using += (incremental update) rather than SET wallet = new_value prevents lost update race conditions. If not wrapped in a transaction, a crash between the two UPDATES would leave the ride completed but driver unpaid.

9. Analytics & Reporting

Q82.

Write SQL: average fare per hour for last 7 days.

Answer:

```
SELECT DATE_TRUNC('hour', requested_at) AS hour, AVG(final_fare_pkr) AS avg_fare, COUNT(*) AS ride_count FROM rides WHERE requested_at >= NOW() - INTERVAL '7 days' AND final_fare_pkr IS NOT NULL GROUP BY hour ORDER BY hour DESC; — Uses `ix_rides_requested_at` index for the WHERE clause.
```

Q83.

How to calculate driver utilisation percentage from raw rides data?

Answer:

Utilisation = (time driver was on active ride) / (time driver was online).
SELECT driver_id, SUM(EXTRACT(EPOCH FROM (completed_at - driver_assigned_at))) / SUM(EXTRACT(EPOCH FROM (last_seen_at - first_online_at))) * 100 AS utilisation_pct FROM rides WHERE status='completed' GROUP BY driver_id; — Requires tracking online/offline events, which the current schema doesn't fully capture.

Q84.

SQL: top 3 most-requested hospitals this month.

Answer:

```
SELECT h.name, COUNT(r.id) AS ride_count FROM rides r JOIN hospitals h ON r.hospital_id = h.id WHERE r.requested_at >= DATE_TRUNC('month', NOW()) GROUP BY h.id, h.name ORDER BY ride_count DESC LIMIT 3;
```

Q85.

How to detect demand spikes using the analytics table?

Answer:

```
SELECT snapshot_hour, total_rides, LAG(total_rides) OVER (ORDER BY snapshot_hour) AS prev_hour_rides, total_rides - LAG(total_rides) OVER (ORDER BY snapshot_hour) AS spike FROM analytics_hourly WHERE city='Islamabad' ORDER BY snapshot_hour DESC; — Hours where spike > 2*avg indicate surge. Can feed a dynamic pricing or driver incentive system.
```

Q86.

SQL: rides where driver took more than 15 minutes to reach patient.

Answer:

```
SELECT id, driver_id, patient_id, driver_assigned_at, pickup_at, EXTRACT(EPOCH FROM (pickup_at - driver_assigned_at))/60 AS minutes_to_pickup FROM rides WHERE pickup_at IS NOT NULL AND driver_assigned_at IS NOT NULL AND pickup_at - driver_assigned_at > INTERVAL '15 minutes' ORDER BY minutes_to_pickup DESC;
```

Q87.

How to generate monthly revenue report per driver.

Answer:

```
SELECT d.full_name, d.license_no, DATE_TRUNC('month', r.completed_at) AS month, COUNT(r.id) AS total_rides, SUM(r.final_fare_pkr) AS total_revenue, AVG(r.final_fare_pkr) AS avg_fare FROM
```

```
rides r JOIN drivers d ON r.driver_id = d.id WHERE r.status = 'completed' GROUP BY d.id,
d.full_name, d.license_no, month ORDER BY month DESC, total_revenue DESC;
```

Q88.

What is `missed\_appointments` in `analytics\_hourly`? How to populate?

Answer:

Scheduled rides that were not completed (cancelled or still pending past scheduled_for time). Cron job SQL: INSERT INTO analytics_hourly (snapshot_hour, city, missed_appointments, ...) SELECT DATE_TRUNC('hour', scheduled_for), 'Islamabad', COUNT(*) FROM rides WHERE ride_type='scheduled' AND scheduled_for < NOW() - INTERVAL '1 hour' AND status IN ('pending','cancelled') GROUP BY 1, 2 ON CONFLICT (snapshot_hour, city, zone_id) DO UPDATE SET missed_appointments = EXCLUDED.missed_appointments;

Q89.

SQL: average fare per ride type.

Answer:

```
SELECT ride_type, COUNT(*) AS total_rides, AVG(final_fare_pkr) AS avg_fare_pkr,
MIN(final_fare_pkr) AS min_fare, MAX(final_fare_pkr) AS max_fare FROM rides WHERE status =
'completed' GROUP BY ride_type;
```

10. Scalability & Future Design

Q90.

How would you partition `rides` by `requested\_at` month?

Answer:

ALTER TABLE rides RENAME TO rides_old; CREATE TABLE rides (LIKE rides_old INCLUDING ALL) PARTITION BY RANGE (requested_at); CREATE TABLE rides_2025_01 PARTITION OF rides FOR VALUES FROM ('2025-01-01') TO ('2025-02-01'); — Each month is a separate physical table. Queries with WHERE requested_at BETWEEN x AND y only scan relevant partitions. New partitions created monthly via cron.

Q91.

If app scales to 1M rides/day, which table is the bottleneck?

Answer:

`rides` accumulates 1M rows/day = 365M/year. `triage\_events` and `family\_notifications` add 3-4x that. `drivers.current\_lat/current\_lng` gets updated every 5s per active driver — at 1000 concurrent drivers, that's 12,000 writes/minute. Consider: 1) Redis for live driver positions (in-memory, no disk writes). 2) Partition rides by month. 3) Time-series DB for analytics_hourly.

Q92.

How would you implement soft deletes?

Answer:

Add `deleted\_at TIMESTAMPTZ DEFAULT NULL` to relevant tables. UPDATE rides SET deleted_at = NOW() instead of DELETE. All queries add WHERE deleted_at IS NULL. Add partial index: CREATE INDEX ON rides(id) WHERE deleted_at IS NULL for query performance. Benefits: audit trail, GDPR erasure via nullifying PII fields, accidental deletion recovery. Cost: queries must always include the filter.

Q93.

What is database sharding? Would it apply here?

Answer:

Sharding splits data across multiple DB servers (e.g., by city: Islamabad shard, Lahore shard). Each shard handles a subset. Benefits: horizontal scale beyond single-server limits. Downsides: cross-shard queries (e.g., national analytics) become complex, no foreign key integrity across shards, operational complexity. For a Pakistan pilot with 2 cities, sharding is premature. Consider at 10+ cities with millions of daily rides.

Q94.

How would you cache hospital lists to reduce DB load?

Answer:

Hospitals are read-heavy, write-rarely (maybe 1 update/week). Cache strategy: 1) On startup, load all hospitals into Redis as JSON. 2) API reads from Redis. 3) On hospital update (admin action), invalidate and reload cache. TTL: 1 hour as fallback. This reduces hospital queries from DB hits to Redis lookups (~0.1ms vs 5ms), important when every ride request queries hospitals.

Q95.

How to support multiple cities with schema changes?

Answer:

Add `city` VARCHAR(100) column to rides and drivers. Add city-based indexes. Seed hospitals per city. Update dispatch logic to filter drivers by city. Update analytics_hourly (already has city column). Optional: add a `cities` reference table with coverage polygon (PostGIS) to auto-detect city from GPS coordinates. No structural schema changes needed — city is an attribute on existing entities.

Q96.

How to add ride history versioning (every status change tracked)?

Answer:

Create `ride\_status\_history` table: (id UUID PK, ride_id FK, old_status ride_status, new_status ride_status, changed_by UUID FK users, changed_at TIMESTAMPTZ). Add a trigger on rides that INSERTs into this table on every status UPDATE. This provides a complete audit trail of every state transition for debugging, disputes, and analytics.

Q97.

What caching would reduce DB load for live driver locations?

Answer:

Driver location updates every 5 seconds — too frequent for DB writes at scale. Use Redis Geo: GEOADD drivers:available . Query nearest: GEORADIUS drivers:available 10 km ASC COUNT 5. Redis Geo is $O(N+\log(M))$ for radius queries. DB location updated asynchronously every 30s for persistence. This offloads ~12,000 writes/minute to in-memory Redis.

Q98.

How would you archive old completed rides?

Answer:

Strategy: 1) Create `rides\_archive` with identical schema. 2) Monthly cron: INSERT INTO rides_archive SELECT * FROM rides WHERE status='completed' AND completed_at < NOW() - INTERVAL '1 year'. 3) DELETE FROM rides WHERE id IN (SELECT id FROM rides_archive WHERE completed_at < NOW() - INTERVAL '1 year'). 4) Optionally move to cold storage (S3 Parquet). Keep rides table hot, archive for compliance/reporting.

Q99.

Explain read/write pattern. Would read replicas help?

Answer:

Write-heavy: driver location updates, ride status transitions, triage event inserts. Read-heavy: pending rides list (all drivers polling), hospital search, patient ride history. Read replicas: sync DB writes to replica, route SELECTs there. Benefits: reduces load on primary, scales read throughput. Caveat: replication lag means driver dashboard might see slightly stale pending rides list — acceptable for NEMT. Use replicas for analytics, reports, hospital queries.

11. ML / Triage Integration

Q100.

What is ``confidence_score`` precision `NUMERIC(5,4)`?

Answer:

`NUMERIC(5,4)` stores up to 5 total digits with 4 after decimal — range 0.0000 to 9.9999, effectively 0.0000 to 1.0000 for a probability score. This gives 4 decimal places of precision (e.g., 0.9873). Sufficient for ML confidence scores. `FLOAT` would introduce floating-point imprecision; `NUMERIC` is exact decimal arithmetic.

Q101.

What does ``rule_override`` mean? Who sets it?

Answer:

``rule_override` = TRUE` means a human (doctor, dispatcher, admin) manually overrode the ML model's specialty prediction. E.g., model predicts 'cardiology' but doctor on call sets it to 'neurology' based on additional info. This flag allows post-hoc analysis: compare cases where override was needed to identify model weaknesses. Set via admin/dispatcher API endpoint.

Q102.

Why is ``model_version`` stored in each triage event?

Answer:

ML models are retrained and updated over time. Storing the version per event allows: 1) Performance analysis — which version was more accurate? 2) Audit trail — what model made this decision? 3) Reproducibility — recreate the prediction context. 4) A/B testing — compare two model versions on real traffic. Without version tracking, historical triage events lose their interpretability.

Q103.

How is ``inference_ms`` useful?

Answer:

``inference_ms`` tracks how long the ML model took to produce a prediction. Useful for: 1) SLA monitoring — alert if inference > 500ms on emergency rides. 2) Performance regression detection after model updates. 3) Infrastructure sizing — average inference time informs GPU/CPU requirements. 4) Identifying problematic inputs that cause slow inference.

Q104.

Can a single ride have multiple triage events?

Answer:

Yes — one-to-many relationship. Scenarios: 1) Patient updates symptoms after initial assessment. 2) Rule override creates a new event. 3) Model re-run after additional vitals provided. 4) Doctor requests re-triage mid-ride. The latest triage event (by `created_at`) is the authoritative prediction. Historical events are retained for audit and model improvement.

Q105.

Data flow from triage to hospital selection.

Answer:

1) Patient submits symptoms → `POST /triage` with `ride_id`. 2) ML model runs → returns `predicted_specialty` + severity_level. 3) triage_events row inserted. 4) Service queries hospitals:`

SELECT * FROM hospitals WHERE specialties @> '['cardiology']' AND is_active=TRUE ORDER BY location <-> pickup_point LIMIT 3. 5) Top hospital assigned to ride (rides.hospital_id updated). 6) Hospital alert sent via hospital_alerts.

Q106.

If ML model retrained, how are old triage predictions affected?

Answer:

Old predictions are immutable — stored with their original model_version. Retraining creates a new version string (e.g., 'v2.1.0'). New rides use the new model. Old triage_events retain their original predictions and model_version. This is correct behavior — historical predictions should not be retroactively changed. Analysis comparing v1 vs v2 performance is done by grouping triage_events by model_version.

12. Real-Time & WebSocket

Q107.

How does driver location update work at DB level? What is the write frequency risk?

Answer:

Driver app sends GPS update every 5 seconds → PATCH /drivers/location → UPDATE drivers SET current_lat=\$1, current_lng=\$2, last_seen_at=NOW() WHERE user_id=\$3. Trigger fires to update GEOGRAPHY column. Risk: at 100 active drivers, that's 100 UPDATE/5s = 20 writes/second to the drivers table. At 1000 drivers: 200 writes/second. Mitigation: buffer in Redis, write to DB every 30s instead of every 5s.

Q108.

Store every GPS ping or only last known location?

Answer:

Store only last known location in DB (current_lat/current_lng). GPS pings are ephemeral — the old position has no value once superseded. Benefits: fixed table size for drivers, fast UPDATE vs INSERT. Use case for storing all pings: post-trip route replay, dispute resolution, billing by distance. If needed, add a `driver\_location\_history` time-series table with TTL (auto-delete after 30 days).

Q109.

What is `last\_seen\_at` in drivers? How is it updated?

Answer:

`last\_seen\_at` records the last time a driver's app was active (sent any API request or WebSocket ping). Updated on every location update or heartbeat. Used for: 1) Detecting ghost drivers — online status=available but no ping for 5+ minutes → auto-set offline. 2) Analytics on driver activity hours. 3) Debugging stale driver positions on the dispatch map.

Q110.

How does `fcm\_token` support push notifications? What if driver changes phones?

Answer:

Firebase Cloud Messaging uses the token to route push notifications to a specific device. When the driver logs in on a new phone, Firebase issues a new FCM token. The app sends it to POST /auth/me (or a dedicated endpoint) → users.fcm_token updated. Old token becomes invalid — Firebase returns 404 on delivery attempts, allowing cleanup. Multiple device support would require a `fcm\_tokens` table (one-to-many).

Q111.

If a driver goes offline mid-ride, what DB state should the ride be left in?

Answer:

The ride should remain in its current status (e.g., driver_en_route or patient_picked_up) — do NOT auto-cancel. Rationale: the driver may have a brief connectivity loss. Actions: 1) Set driver.status = 'offline'. 2) Alert dispatcher/admin via hospital_alerts or internal notification. 3) Start a grace period timer (5 min). 4) After grace period with no reconnection, flag ride for manual intervention. Auto-cancelling would leave patients stranded.
